

ICS321 – Database Systems

Horse Racing Database Project



Mohammed Alfaraj – 202323090 – Section 1

Redha Alturaik – 202323010 – Section 3

Group: Project Team 46

Date: 11/3/2025

Term 251

Tools and Resources Used

Tools:

- Visual Studio 2022
- NuGet Package (MySQL.Data)
- MySQL Server and Workbench 8.0
- Programming Languages used: C# and HTML

Resources:

- Zybook
- MySQL Documentation
- Microsoft ASP.NET Documentation
- W3School

Problem Faced and Resolved

- 1- MySQL not connecting with the visual Studio.
 - Resolved: We had to install a library in the project called MySQL.Data to use the database in the MySQL server.
- 2- Results not showing in the gridview in the webpage.
 - Resolved: We made sure that every column name and gridview ID matched to ensure that all data is taken from the table and is displayed in the appropriate gridview.

Functionality Implemented

Guest Side

1- Select horses Based on owner's last name:

- Allows the user to display the horse's name, horse age, and the trainer's first and last name as one column, based on the owner's last name the user enters. It uses 3 joins with the owns, owner, and Trainer tables to get the result, and a where clause to lowercase both names to ensure the name in the database is the same as the inserted name by the user, and the data is ordered by horse age descending.

2- Show trainers who have raised winning horses:

- Allows the user to check the trainers who raised winning horses by displaying the trainer's first name, trainer's last name, horse name, race name, track name, race data with DATE_FORMAT to show the date accordingly, and race time. It uses 3 joins with the Horse, RaceResults, and Race tables and using a where clause to check if the horse got first place, to find the trainers and display them to the user, and the data is ordered by trainer's first name and then trainer's last name.

3- Show trainer total amount of winning in term of prize money:

- Allows the user to check the trainer total winning earned from prize money by displaying trainer's first name, trainer's last name, and winnings. It uses 2 left joins to ensure all trainers are shown even if they do not have any earning, and they join with Horse and

RaceResults, and uses group by trainers to stop duplicates of trainer's name, and order by to make the order of the record appear descending based on winnings descending.

4- Show tracks and the number of races and horses on the track:

- It allows the user to show the tracks and their details by displaying the track name, number of races, and total horses present. It uses one left join to show races with null records. It also uses group by the track name to make them distinct, and ordered by to make the records appear ascending based on track name.

Admin Side:

1- Add a new race with the results:

- This function allows the admin to create a new race and record its outcomes. The admin provides a race ID, race name, track name, race date, and race time. The system checks that all fields are filled correctly the date must follow the "yyyy-MM-dd" format and the time must follow "HH:mm:ss". Once validated, the system inserts the race information into the Race table. After adding the race, the admin can enter the race results, selecting the race and horse, then recording the finishing position and prize amount. These details are stored in the RaceResults table, which links each horse to a race and its result. This ensures both the race and its results are properly documented.

2- Delete an owner and all their details:

- This function removes an owner and their related information from the system. The admin selects or specifies the owner's ID to delete. The system deletes the corresponding record from the Owner table. If the database has foreign key relationships with cascade delete enabled, any related records such as ownership links in the Owns table are automatically deleted as well. If not, the admin would need to manually remove any dependent data. Once the deletion is complete, the interface refreshes to update the list of owners and related data, ensuring the owner's details are completely removed from the database.

3- Move a horse from one stable into another based on their horse ID:

- This function lets the admin reassign a horse from one stable to another. The admin enters the horse's unique ID and selects the destination stable from a dropdown menu. The system verifies both the horse ID and stable selection before proceeding. When confirmed, the system updates the Horse table, setting the stableId field to the new stable's ID. This change immediately updates the horse's stable assignment in the database. Afterward, all related data displays are refreshed so the horse now appears under its new stable.

4- Approving a new trainer to a stable:

- This function allows the admin to approve and register a new trainer to a specific stable. The admin provides a trainer ID, first name, last name, and chooses the stable the trainer will join. The system validates that all required fields are completed. It then inserts a new record into the Trainer table, linking the trainer to the selected stable using the stableId. This process effectively approves the trainer, adding them as an official member of the stable's team. After approval, the trainer's information becomes visible in the system under the associated stable.

Tasks Done by Each Group Member

Mohammed Alfaraj: Worked on the design of the guest side which includes both the UI and the backend, which consists of the methods listed for the guest such as, selecting horses based on the last name, trainers who raised winning horses, trainers total earning, and track details.

Redha Alturaik: Worked on the design of the admin side, both the UI and the backend code, and procedure code. Which consists of adding a new race with the results, deleting an owner with their information from the database, using a horse ID to change a horse stable into another one, and approving a new trainer to join a stable. And to copy all horse information to an old info table when a horse is deleted.

Appendix A

Guest Side:

Select horses Based on owner's last name:

```
Select Horse.horseName AS 'Horse Name', Horse.age AS 'Age', concat(Trainer.fname, ' ',  
Trainer.lname) AS 'Trainer Name'  
From Horse  
Join owns on Horse.horseId = owns.horseId  
Join owner on owns.ownerId = owner.ownerId  
Join Trainer on horse.stableId = Trainer.stableId  
Where lower(owner.lname) = lower(@SelectedName)  
Order By Horse.age desc;
```

Assumption: The task from what I understood said to search for the names of both the horse and the trainer, and the horse's age based on the last name entered. I ordered it with horse's age, descending, to make it easier to see the horses from oldest to youngest, and the where clause to ensure that the entered name would match the name in the database regardless of capitalizing letters or not.

Show trainers who have raised winning horses:

```
Select Trainer.fname AS 'Trainer First Name', Trainer.lname AS 'Trainer Last Name',  
Horse.horseName AS 'Winning Horse', Race.raceName AS 'Race Name',  
DATE_FORMAT(Race.raceDate, '%Y-%M-%D') AS 'Race Date', Race.raceTime AS  
'Race Time'  
From Trainer  
Join Horse On Trainer.stableId = Horse.stableId  
Join RaceResults On Horse.horseId = RaceResults.horseId  
Join Race On RaceResults.raceId = Race.raceId  
Where RaceResults.results = 'first'  
Order By Trainer.fname, Trainer.lname;
```

Assumption: The task from what I understood asks to show the trainer's first name and last name as separate columns, the winning horse's name, and the race with details to all of them. This included the race date and time with the other specified columns to show the time and date of the horse's win and the trainer who raised the horse. The data is ordered by the first name of the trainer, and then last name of the trainer to make it easier to see if a trainer has raised multiple winning horses.

Show trainer total amount of winning in term of prize money:

```

Select Trainer.fname AS 'Trainer First Name', Trainer.lname AS 'Trainer Last Name',
IFNULL(SUM(RaceResults.prize), 0) AS 'Winnings'
From Trainer
Left Join Horse On Trainer.stableId = Horse.stableId
Left Join RaceResults On Horse.horseId = RaceResults.horseId
Group By Trainer.fname, Trainer.lname
Order By `Winnings` desc;

```

Assumption: The task from what I understood asks to check the first and last name of the trainer, both in separate columns, and their total winnings in term of prize money, and sort them based on winning. I used a left join to show trainers who do not have any money registered since it asked to check all trainers. The data is ordered descending by winning to show the highest earning trainer first.

Show tracks and the number of races and horses on the track:

```

select Race.trackName as 'Track Name', COUNT(DISTINCT Race.raceId) AS 'Number
of Races', COUNT(RaceResults.horseId) as 'Total Horses'
From Race
Left Join RaceResults on race.raceId = RaceResults.raceId
Group By race.trackname
Order By race.trackName;

```

Assumption: The task from what I understood asks to show all tracks and the number of races held, and horses present in each track. So, track will be the first column. The second column will be the number of races, and I used distinct race ID to check the number of races without any duplicates. And for the third and last column is the number of horse, and similarly, I used distinct on the horse ID to check the number of horses present in each track without any duplicates. The data is grouped by track name to list each track once, and ordered by track name to show each track alphabetically.

Admin Side:

Add a new race with the results:

```
0 references
protected void btnAddRace_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtRaceId.Text)) { lblToast.Text = "Race ID is required."; return; }
    if (string.IsNullOrEmpty(ddlRaceTrack.SelectedValue)) { lblToast.Text = "Select a track."; return; }

    if (!DateTime.TryParseExact(txtRaceDate.Text.Trim(), "yyyy-MM-dd", CultureInfo.InvariantCulture,
        DateTimeStyles.None, out var date))
    { lblToast.Text = "Date must be yyyy-MM-dd."; return; }

    if (!TimeSpan.TryParseExact(txtRaceTime.Text.Trim(), "hh\\:\\:mm\\:\\:ss", CultureInfo.InvariantCulture,
        TimeSpanStyles.None, out var time))
    { lblToast.Text = "Time must be HH:mm:ss."; return; }

    Exec(@"INSERT INTO Race (raceId, raceName, trackName, raceDate, raceTime)
VALUES (@id,@n,@t,@d,@tm)",
        P("@id", txtRaceId.Text.Trim()),
        P("@n", txtRaceName.Text.Trim()),
        P("@t", ddlRaceTrack.SelectedValue),
        PT("@d", MySqlDbType.Date, date.Date),
        PT("@tm", MySqlDbType.Time, time));

    lblToast.Text = $"Race added (ID: {txtRaceId.Text}).";
    txtRaceId.Text = txtRaceName.Text = txtRaceDate.Text = txtRaceTime.Text = string.Empty;
    LoadLookups();
    LoadAll();
}
```

Assumption:

From what I understood, this function allows the user to add a new race record into the database by providing the race ID, name, selected track, date, and time. It first validates that all fields are filled and that the date and time follow the correct formats (yyyy-MM-dd for date and HH:mm:ss for time). If everything is valid, it executes an INSERT statement into the Race table with those values. The function also updates the display to confirm the race was added and refreshes the data. The input validation helps prevent bad data from being stored.

Delete an owner and all their details:

```
0 references
protected void gvOwners_RowDeleting(object sender, GridViewDeleteEventArgs e)
{
    string id = gvOwners.DataKeys[e.RowIndex].Value.ToString();
    Exec("DELETE FROM Owner WHERE ownerId=@id", P("@id", id));
    lblToast.Text = "Owner deleted.";
    LoadAll();
}
```

Assumption:

From what I understood, this function deletes an owner from the Owner table using their ownerId. The function retrieves the owner's ID from the selected row in a grid view and executes a DELETE SQL command. Once completed, it confirms deletion to the user through a message label and reloads all remaining data. The goal is to completely remove the owner and any related details from the system.

Move a horse from one stable into another based on their horse ID:

```
0 references
protected void btnMoveHorse_Click(object sender, EventArgs e)
{
    var horseId = txtMoveHorseId.Text.Trim();
    if (string.IsNullOrEmpty(horseId))
    { lblToast.Text = "Enter a Horse ID like 'horse12'."; return; }
    if (string.IsNullOrEmpty(ddlMoveStable.SelectedValue))
    { lblToast.Text = "Choose the new stable."; return; }

    int affected = Exec("UPDATE Horse SET stableId=@s WHERE horseId=@h",
        P("@s", ddlMoveStable.SelectedValue), P("@h", horseId));

    lblToast.Text = affected > 0 ? "Horse moved to the new stable." : "No horse found with that ID.";
    LoadAll();
}
```

Assumption:

From what I understood, this function updates a horse's stable assignment. It takes a horseId input and a selected new stable, checks that both values are valid, and then performs an UPDATE query on the Horse table to change the stableId. If the update affects any rows, it confirms that the horse was successfully moved; otherwise, it alerts that no horse with that ID was found. It also reloads the display to show the updated stable assignments. This ensures accurate management of horses' stable locations.

Adding a new stable to a stable:

```
0 references
protected void btnAddStable_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtStableId.Text))
    { lblToast.Text = "Stable ID is required (e.g., stable9)."; return; }

    Exec("INSERT INTO Stable (stableId, stableName, location, colors) VALUES (@id,@n,@loc,@c)",
        P("@id", txtStableId.Text.Trim()), P("@n", txtStableName.Text.Trim()),
        P("@loc", txtStableLocation.Text.Trim()), P("@c", txtStableColors.Text.Trim()));
    lblToast.Text = $"Stable added (ID: {txtStableId.Text}).";
    txtStableId.Text = txtStableName.Text = txtStableLocation.Text = txtStableColors.Text = string.Empty;
    LoadLookups();
    LoadAll();
}
```

Assumption:

From what I understood, this function lets the user add a new stable record into the Stable table. It validates that a stable ID is entered, then executes an INSERT command using the entered stable ID, name, location, and color values. Once the data is inserted, it confirms the addition and clears the input fields. The display and lookup lists are then refreshed. This ensures that users can expand the database with new stable information.

Adding a new trainer to a stable

```
0 references
protected void btnAddTrainer_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtTrainerId.Text))
    { lblToast.Text = "Trainer ID is required (e.g., trainer11)."; return; }
    if (string.IsNullOrEmpty(ddlTrainerStable.SelectedValue))
    { lblToast.Text = "Select a stable for the trainer."; return; }

    Exec("INSERT INTO Trainer (trainerId, fname, lname, stableId) VALUES (@id,@f,@l,@s)",
        P("@id", txtTrainerId.Text.Trim()), P("@f", txtTrainerFName.Text.Trim()),
        P("@l", txtTrainerLName.Text.Trim()), P("@s", ddlTrainerStable.SelectedValue));
    lblToast.Text = $"Trainer added (ID: {txtTrainerId.Text}).";
    txtTrainerId.Text = txtTrainerFName.Text = txtTrainerLName.Text = string.Empty;
    LoadAll();
}
```

From what I understood, this function adds a new trainer to an existing stable. It checks that both a trainer ID and a stable selection are provided. Once validated, it performs an INSERT query to the Trainer table with the trainer's ID, first name, last name, and stable ID. After successful insertion, it shows a confirmation message and clears the input fields. The method helps maintain accurate records of trainers and their assigned stables.

Procedures:

USE racing;

Create stored procedure to delete an owner and related data

```
DELIMITER //
CREATE PROCEDURE delete_owner_and_related(IN p_ownerId VARCHAR(20))
BEGIN
    DELETE FROM Owns WHERE ownerId = p_ownerId;
    DELETE FROM RaceResults
    WHERE horseId IN (SELECT horseId FROM Owns WHERE ownerId = p_ownerId);
    DELETE FROM Owner WHERE ownerId = p_ownerId;
END //
DELIMITER ;
```

Create table to store deleted horse info

```
CREATE TABLE IF NOT EXISTS old_info (
    horseId VARCHAR(20),
    horseName VARCHAR(50),
    age INT,
    gender VARCHAR(10),
    registration INT,
    stableId VARCHAR(20),
    deleted_at DATETIME
);
```

Create trigger to copy horse info before deletion

```
DELIMITER //
CREATE TRIGGER trg_copy_horse_before_delete
BEFORE DELETE ON Horse
FOR EACH ROW
BEGIN
    INSERT INTO old_info (horseId, horseName, age, gender, registration, stableId, deleted_at)
    VALUES (OLD.horseId, OLD.horseName, OLD.age, OLD.gender, OLD.registration,
    OLD.stableId, NOW());
END //
DELIMITER ;
```